

Лабораторна робота № 1. Розгортання середовища та порівняльний аналіз мовних моделей

Мета й очікуваний результат

Після роботи студент повинен:

- запустити мовну модель локально й керувати нею з коду;
- розуміти, що локальна модель і хмарна — це один і той самий інтерфейс, і вміти перемикатися між ними зміною двох рядків;
- вимірювати продуктивність інференсу й пояснювати, з чого складається затримка;
- емпірично показати вплив `temperature` і `top_p` на стабільність відповідей.

Хід роботи

Крок 1. Встановити Ollama і завантажити моделі

Мовна модель (LLM) — це файл на кілька гігабайтів. Сам по собі він нічого не робить. Щоб він працював, потрібна програма, яка його завантажить у пам'ять і буде «крутити». У цій лабораторній роботі використовується вже готова модель, яку потрібно запустити локально.

Ollama — це програма і локальний сервіс для запуску мовних моделей на комп'ютері. Вона дозволяє завантажувати моделі, запускати їх і звертатися до них з інших програм. Вона запускається як фонові служба і починає слухати запити на вашому комп'ютері за адресою `http://localhost:11434`.

Порядок дій:

1. Зайдіть на `ollama.com`, завантажте версію для вашої системи (Windows / macOS / Linux), встановіть звичайним способом. Після встановлення програма працює у фоні.
2. Відкрийте термінал (у Windows — PowerShell або «Командний рядок») і перевірте, що все встановилось:

```
ollama --version
```

Якщо бачите номер версії — все добре. Якщо «команда не знайдена» — перезапустіть термінал.

3. Завантажте моделі. Кожна команда качає кілька гігабайтів, тому переконайтеся, що у вас є вільне місце на диску:

```
ollama pull qwen3:4b  
ollama pull llama3.2:3b
```

Тут `qwen3` — назва сімейства моделей, `4b` — 4 мільярди параметрів, тобто «розмір мозку». Більше параметрів — розумніша, але повільніша й потребує більше оперативної пам'яті.

Якщо у вас 16 GB RAM або більше, додайте ще й більшу модель:

```
ollama pull qwen3:8b
```

Якщо RAM 8 ГБ — не завантажуйте, комп'ютер буде зависати, працюйте з двома моделями.

4. Перевірте, що завантажилось:

```
ollama list
```

Ви побачите список моделей та їхній розмір на диску.

5. Виконайте запит до моделі вручну, щоб переконатися, що вона «жива»:

```
ollama run qwen3:4b "Надайте інформацію про вашу модель"
```

Відповідь може з'являтися повільно, буквально по кілька слів за секунду. Це нормально, якщо у вас немає відеокарти — модель рахує на процесорі.

Крок 2. Побачити, що модель – це звичайний веб-сервер

Порядок дій:

1. Виконайте в терміналі запит напряму з використанням моделі `qwen3:4b` :

```
curl http://localhost:11434/api/generate -d '{"model":"qwen3:4b","prompt":"  
\ "Столиця України?", "stream":false}'
```

2. Пояснення полів:

- `response` – власне текст відповіді, який побачить користувач;
- `thinking` – внутрішні роздуми моделі. З'являється лише в міркувальних моделях. Зверніть увагу на обсяг - п'ять абзаців самоперевірки заради однослівної відповіді. Модель кілька разів перепитує саму себе, чи не Львів це, чи не Харків — і щоразу відповідає «ні, точно Київ». Це наочна ілюстрація того, що LLM не «згадує факт», а поступово породжує текст;
- `done: true` — генерація завершена. При стрімінгу проміжні шматочки приходять з `false`, і `true` стоїть лише в останньому;
- `done_reason: stop` — чому зупинилися. Це поле варто перевіряти коді завжди. У наступних лабораторних обірвана відповідь ламатиме агента, а зовні виглядатиме як «просто якась дивна поведінка». Три типові значення:
 - `stop` — модель сама вирішила, що відповідь завершена. Нормальний випадок;
 - `length` — уперлася в ліміт токенів, відповідь обірвана на півслові;
 - `load` — службова зупинка.
- `context` — той самий діалог, але у вигляді номерів токенів (151644, 872, 198, ...);
- `load_duration` - час завантаження моделі в пам'ять (в наносекундах);
- `prompt_eval_duration` - час обробки запиту (в наносекундах);
- `eval_duration` - генерація відповіді (в наносекундах);
- `total_duration` - усе разом (в наносекундах). Сума може не співпадати із-за накладних витрат (розбір JSON, формування відповіді, буферизація тощо);
- `prompt_eval_count` - кількість токенів. Зверніть увагу: сам текст «Столиця України?» — це приблизно 6–7 токенів. Решта — службова розмітка (позначки початку повідомлення, ролі, кінця). Ці невидимі токени теж займають контекст і теж оплачуються в платних API;

- `prompt_eval_cached_count: 3` — три токени з дев'ятнадцяти взято з кешу, їх не довелося обчислювати заново;
- `eval_count: 409` — скільки tokenів згенеровано. Разом з роздумами.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Моделі бувають двох типів:

- інструктивні (наприклад, `llama3.2:3b`) одразу видають відповідь на запит;
- міркувальні (наприклад, `qwen3:4b`) спершу генерують внутрішню чернетку – «хід думок» – і кладуть її в окреме поле `thinking`. Користувач бачить лише кінцеву відповідь у полі `response`. Чернетка допомагає на задачах, де треба щось порахувати або розкласти на кроки, але на простому фактичному питанні лише витрачає токени й час.

Два показники, які не можна плутати:

- tokenів за секунду — характеризує обладнання й розмір моделі;
- загальний час відповіді — характеризує обсяг роботи, яку модель собі призначила.

Крок 1. Просте фактичне питання. Виконати для обох моделей:

```
curl http://localhost:11434/api/generate -d '{"model\":"qwen3:4b",
  "prompt\":"Столиця України? Відповідай одним словом.\",\"stream\":false}'
```

```
curl http://localhost:11434/api/generate -d '{"model\":"llama3.2:3b",
  "prompt\":"Столиця України? Відповідай одним словом.\",\"stream\":false}'
```

Крок 2. Задача на міркування. Виконати для обох моделей по 5 разів:

```
curl http://localhost:11434/api/generate -d '{"model\":"qwen3:4b",
  "prompt\":"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша
  за зошит. Скільки коштує ручка?\",\"stream\":false}'
```

```
curl http://localhost:11434/api/generate -d '{"model\":"llama3.2:3b",
  "prompt\":"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша
  за зошит. Скільки коштує ручка?\",\"stream\":false}'
```

Правильна відповідь — 5 грн.

Крок 3. Порахувати (тривалості в JSON — наносекунди):

```
час, c    = total_duration / 1e9
tokenів/c = eval_count / (eval_duration / 1e9)
```

Крок 4. Заповнити таблиці:

Таблиця 1: Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response		
eval_count		
Час, с		
Токенів/с		

Таблиця 2: Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5		
eval_count		
Час, с		

Крок 6. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць кроку 4 – назви моделей, eval_count, час, кількість правильних відповідей із 5 – і поставити п'ять питань:

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однаково, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?
2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?
3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?
4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?
5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b ?

У звіт додати дані таблиць та стислі відповіді на питання (2–3 речення на кожному). Стисло напишіть свої думки стосовно особливостей використання міркувальних та інструктивних моделей.

Крок 3. Керувати моделлю з програми на Python

Порядок дій:

1. Встановіть Python. Спочатку перевірте, чи встановлений Python (виконайте в терміналі):

```
python --version
```

Якщо команда показує версію Python, наприклад Python 3.14.7, значить Python встановлений. Якщо система пише, що команда не знайдена, Python потрібно встановити з офіційного сайту:

```
https://www.python.org/downloads/
```

Для цієї лабораторної роботи бажано використовувати Python 3.8 або новіший.

2. Створіть папку проєкту та віртуальне середовище:

```
mkdir lab1 && cd lab1
python -m venv venv
```

Активуйте його:

```
venv\Scripts\activate.bat
```

Ознака успіху – на початку рядка з'явиться (venv).

3. Встановіть бібліотеку `openai` :

OpenAI-сумісний API — компанія OpenAI свого часу задала формат таких запитів, і він став галузевим стандартом. Тепер майже всі — і Ollama, і Google, і Groq — приймають запити в тому самому форматі. Практичний наслідок: студент пише код один раз, а потім перемикається між локальною моделлю і хмарною, змінивши одну адресу.

```
pip install openai python-dotenv
```

Бібліотека називається `openai`, але це не означає, що ви користуєтесь OpenAI. Це просто клієнт, який вміє говорити в тому стандартному форматі, який розуміє і ваша Ollama.

4. Створіть файл `hello.py` :

```
from openai import OpenAI

# Кажемо клієнту стукати не в OpenAI, а на наш комп'ютер
client = OpenAI(
    base_url="http://localhost:11434/v1", # адреса локальної Ollama
    api_key="ollama",                    # Ollama ключ не перевіряє,
                                        # але поле обов'язкове
)

response = client.chat.completions.create(
    model="qwen3:4b",
    messages=[
        {"role": "system", "content": "Ти лаконічний технічний асистент."},
        {"role": "user", "content": "Поясни різницю між списком і кортежем у Python."},
    ],
    temperature=0.7,
)

print(response.choices[0].message.content)
print(response.usage) # скільки токенів витрачено
```

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Модель не «пам'ятає» розмову — при кожному виклику ви передаєте їй усю історію в параметрі `messages`. Це список повідомлень, кожне з роллю:

- `system` — інструкція, яка задає поведінку моделі: мову, стиль, довжину, формат відповіді. Користувач її не бачить, але вона діє на всю розмову.
- `user` — те, що пише людина.
- `assistant` — те, що відповіла модель. З'являється, коли ви ведете діалог з кількох реплік і передаєте попередні відповіді назад.

Практичний наслідок: одна й та сама модель на один і той самий запит дає різні відповіді залежно від `system`-повідомлення. Це найдешевший спосіб керувати моделлю — не потрібно ні донавчання, ні зміни коду. Різні моделі по-різному слухаються цих інструкцій.

Варіанти (номер у журналі):

Питання користувача (`user`):

1. Поясни різницю між списком і кортежем у Python.
2. Поясни різницю між стеком і чергою.
3. Поясни різницю між компілятором та інтерпретатором.
4. Поясни різницю між оперативною пам'яттю і жорстким диском.
5. Поясни різницю між масивом і зв'язним списком.
6. Поясни різницю між HTTP і HTTPS.
7. Поясни різницю між процесом і потоком.
8. Поясни різницю між реляційною і документоорієнтованою базою даних.
9. Поясни різницю між функцією і методом.
10. Поясни різницю між синхронним і асинхронним виконанням коду.

Крок 1. Створити файл `hello.py`:

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1", # адреса локальної Ollama
    api_key="ollama",                  # Ollama ключ не перевіряє,
                                     # але поле обов'язкове
)

SYSTEM = "Ти лаконічний технічний асистент."
USER = "<питання вашого варіанта>"

response = client.chat.completions.create(
    model="llama3.2:3b",
    messages=[
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": USER},
    ],
    temperature=0.7,
)
```

```
print(response.choices[0].message.content)
print(response.usage)
```

Крок 2. Запустити: `python hello.py` . Зберегти відповідь.

Крок 3. Змінити system-повідомлення на:W

```
SYSTEM = "Відповідай лише українською, максимум двома реченнями."
```

Запустити знову, зберегти відповідь.

Крок 4. Повторити кроки 1–3 для моделі qwen3:4b (змінити model у коді).

Крок 5. Заповнити таблиці (дані брати з response.usage).

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент				
llama3.2:3b	українською, 2 речення				
qwen3:4b	лаконічний асистент				
qwen3:4b	українською, 2 речення				

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент			
llama3.2:3b	українською, 2 речення			
qwen3:4b	лаконічний асистент			
qwen3:4b	українською, 2 речення			

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п'ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?
2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?
3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?
4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?
5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

У звіт додати дані таблиць та стислі відповіді на питання (2–3 речення на кожному). Стисло напишіть свої думки стосовно ролі system-повідомлення: наскільки надійно ним можна керувати поведінкою моделі, чи достатньо самої інструкції для отримання потрібного формату відповіді, і що варто робити, якщо модель її ігнорує.

Крок 4. Підключити хмарну модель

Той самий код працює і з локальною моделлю, і з чужим сервером на іншому континенті. Змінюються тільки адреса й ключ. Саме завдяки цьому в реальних проєктах можна швидко перемикаати провайдерів, коли один дорожчає, гальмує або лежить.

Хмарна модель значно більша за ту, що розташована на вашому комп'ютері. Ви побачите різницю в якості — і одночасно замислитесь, що ваш текст пішов на чужий сервер.

Порядок дій:

1. Отримати ключ доступу.

Що таке API-ключ. Це довгий рядок символів, який замінює логін і пароль для програм. Коли ваш код надсилає запит на чужий сервер, він додає цей ключ, і сервер розуміє, чий це запит і чи не вичерпано ліміт. Ключ прирівнюється до пароля: хто його має — витрачає ваш ліміт від вашого імені.

Зареєструйтеся в одного з провайдерів з безкоштовним тарифом (картка не потрібна):

- Google AI Studio — найщедріший безкоштовний ліміт, моделі сімейства Gemini Flash;
- Groq — найшвидша генерація, працює на спеціальному «залізі», моделі з відкритою вагою.

У розділі API keys створіть новий ключ і одразу скопіюйте його — більшість сервісів показують ключ лише один раз. Знайдіть у документації провайдера два значення, вони знадобляться в коді:

- `base_url` — адреса, куди слати запити (наприклад, для Groq це `https://api.groq.com/openai/v1`);

- точна назва моделі — рядок на кшталт `llama-3.3-70b-versatile`. Назву треба брати з документації дослівно, з помилкою в одному символі сервер поверне помилку.

2. Приховати ключ від чужих очей

Це не формальність. Найпоширеніша помилка початківців — вписати ключ прямо в код, залити код на GitHub і за кілька годин побачити, що ліміт вичерпано. Спеціальні боти цілодобово сканують публічні репозиторії саме в пошуках ключів. Тому ключ ніколи не пишуть у коді.

У папці проєкту створіть файл з назвою рівно `.env` (з крапкою на початку):

```
CLOUD_API_KEY=тут_ваш_ключ_без_лапок
```

Створіть файл `.gitignore` і додайте туди рядок:

```
.env
venv/
```

`.gitignore` — це список файлів, які Git ігнорує й ніколи не відправляє на GitHub.

Перевірте, що спрацювало. Виконайте:

```
git status
```

Файл `.env` не має з'явитися у списку. Якщо він там є — ви або створили `.gitignore` не в тій папці, або встигли додати `.env` у Git раніше.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Формат запиту, який колись задала компанія OpenAI, став галузевим стандартом. Тепер його приймають майже всі. Практичний наслідок: код пишеться один раз, а перемикання між локальною й хмарною моделлю зводиться до зміни адреси й ключа. Різниця між джерелами при цьому суттєва. Локальна модель безкоштовна, працює без інтернету, і ваші дані нікуди не йдуть — але вона мала й слабка. Хмарна модель у десятки разів більша, відповідає краще — але залежить від мережі, має ліміти й бачить усе, що ви їй надсилаєте.

Варіанти (номер у журналі):

Питання (QUESTION):

1. Поясни різницю між списком і кортежем у Python. Коротко.
2. Поясни різницю між стеком і чергою. Коротко.
3. Поясни різницю між компілятором та інтерпретатором. Коротко.
4. Поясни різницю між оперативною пам'яттю і жорстким диском. Коротко.
5. Поясни різницю між масивом і зв'язним списком. Коротко.
6. Поясни різницю між HTTP і HTTPS. Коротко.
7. Поясни різницю між процесом і потоком. Коротко.
8. Поясни різницю між реляційною і документоорієнтованою базою даних. Коротко.

9. Поясни різницю між функцією і методом. Коротко.
10. Поясни різницю між синхронним і асинхронним виконанням коду. Коротко.

Крок 1. Отримати ключ, base_url та точну назву моделі (див. вище).

Крок 2. Написати код

Файл `providers.py` :

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv() # зчитує файл .env і робить його значення доступними

PROVIDERS = {
    "local": {
        "base_url": "http://localhost:11434/v1",
        "api_key": "ollama",
        "model": "llama3.2:3b",
    },
    "cloud": {
        "base_url": "<адреса провайдера>",
        "api_key": os.getenv("CLOUD_API_KEY"),
        "model": "<точна назва з документації>",
    },
}

def make_client(name):
    """Повертає готовий клієнт і назву моделі для обраного провайдера."""
    cfg = PROVIDERS[name]
    client = OpenAI(base_url=cfg["base_url"], api_key=cfg["api_key"])
    return client, cfg["model"]
```

Всі відмінності між провайдерами зібрані в одному словнику — решта програми про них не знає взагалі.

Файл `compare.py` :

```
import time
from providers import make_client

QUESTION = "<питання вашого варіанта>"

for name in ["local", "cloud"]:
    client, model = make_client(name)

    t0 = time.perf_counter()
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": QUESTION}],
```

```

        temperature=0.7,
    )
    elapsed = time.perf_counter() - t0

    print(f"==== {name.upper()} ({model}) =====")
    print(response.choices[0].message.content)
    print(f"Час: {elapsed:.2f} c")
    print(response.usage)
    print()

```

Крок 3. Запустити код

```
python compare.py
```

Цикл однаковий для обох випадків. Функція `create()` викликається абсолютно ідентично. Змінився лише вміст словника.

Типові помилки:

- `401 Unauthorized` – ключ невірний або не підвантажився. Перевірте `load_dotenv()` і що `.env` лежить у папці, звідки запускаєте скрипт;
- `404 model not found` – неточна назва моделі. Скопіюйте з документації дослівно;
- `429 Too Many Requests` – вичерпано безкоштовний ліміт. Зачекайте або візьміть іншого провайдера;
- `os.getenv` повертає `None` – файл названо `env.txt` замість `.env` – Windows часто дописує розширення.

Крок 4. Заповнити таблицю (дані брати з `response.usage`).

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі		
Час відповіді, c		
<code>completion_tokens</code>		
Довжина відповіді, символів		
Мова відповіді		
Фактичних помилок у відповіді		

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?
2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.
3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?
4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Крок 6. Питання до чат-бота:

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?
2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?
3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?
4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проектах?
5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

У звіт додати дані таблиць та стислі відповіді на питання (2–3 речення на кожному). Стисло напишіть свої думки: у яких випадках ви обрали б локальну модель, попри її слабкість, а в яких — хмарну, попри залежність від мережі й передачу даних назовні.

Крок 5. Вимірювати швидкість для різних провайдерів

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Затримка відповіді складається з двох різних величин, і одним числом її не описати:

- **TTFT (time to first token)** — час до появи першого слова. Поки він триває, користувач бачить порожній екран. Для локальної моделі це обробка вашого запиту, для хмарної — ще й дорога по мережі;
- **Швидкість генерації** — скільки токенів за секунду видається далі. Від неї залежить тривалість довгої відповіді.

Ці показники можуть суперечити один одному: хмарна модель часто має гірший TTFT (пакет летить через пів світу), але кращу швидкість генерації. На короткій відповіді локальна виглядає швидшою, на довгій — програє.

Щоб виміряти TTFT, відповідь треба отримувати по шматочках — це стрімінг, вмикається параметром `stream=True`. Той самий ефект, коли в чат-боті текст «друкується» на очах.

Три правила коректного вимірювання: прогрів (перший запуск не рахуємо), три повтори з медіаною, без паралельних запитів.

Варіанти (номер у журналі):

Середній промпт для всіх однаковий: «Поясни в одному абзаці, що таке рекурсія.»

Таблиця 6: Варіанти промптів

№	Короткий промпт	Довгий промпт
1	Столиця Франції?	Напиши інструкцію з 10 пунктів, як почати вивчати Python.
2	Столиця Японії?	Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.
3	Столиця Канади?	Напиши інструкцію з 10 пунктів, як організувати резервне копіювання даних.
4	Столиця Бразилії?	Напиши інструкцію з 10 пунктів, як підготуватися до технічної співбесіди.
5	Столиця Єгипту?	Напиши інструкцію з 10 пунктів, як почати працювати з Git.
6	Столиця Індії?	Напиши інструкцію з 10 пунктів, як спроектувати просту базу даних.
7	Столиця Норвегії?	Напиши інструкцію з 10 пунктів, як провести код-рев'ю.
8	Столиця Мексики?	Напиши інструкцію з 10 пунктів, як задокументувати проєкт.
9	Столиця Туреччини?	Напиши інструкцію з 10 пунктів, як оптимізувати повільний застосунок.
10	Столиця Австралії?	Напиши інструкцію з 10 пунктів, як розгорнути вебзастосунок на сервері.

Крок 1. Створити файл `benchmark.py`

```

import time
import statistics
import csv
from providers import make_client

def measure(client, model, prompt):
    """Один вимір: повертає TTFT, загальний час і довжину відповіді."""
    t_start = time.perf_counter()
    first_token_at = None
    parts = []

    stream = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )

    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            if first_token_at is None:
                first_token_at = time.perf_counter() # зафіксували перший
            token
            parts.append(piece)

```

```

t_end = time.perf_counter()
text = "".join(parts)

return {
    "ttft_s": round(first_token_at - t_start, 3),
    "total_s": round(t_end - t_start, 3),
    "chars": len(text),
    "chars_per_s": round(len(text) / (t_end - t_start), 1),
}

PROMPTS = {
    "short": "<короткий промпт вашого варіанта>",
    "medium": "Поясни в одному абзаці, що таке рекурсія.",
    "long": "<довгий промпт вашого варіанта>",
}

rows = []
for provider in ["local", "cloud"]:
    client, model = make_client(provider)

    measure(client, model, "позирпів")          # прогнів, не рахуємо

    for name, prompt in PROMPTS.items():
        runs = [measure(client, model, prompt) for _ in range(3)]
        rows.append({
            "provider": provider,
            "model": model,
            "prompt": name,
            "ttft_s": statistics.median(r["ttft_s"] for r in runs),
            "total_s": statistics.median(r["total_s"] for r in runs),
            "chars_per_s": statistics.median(r["chars_per_s"] for r in
runs),
        })
    print(rows[-1])

with open("results.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)

```

`time.perf_counter()` — точний таймер для вимірювання інтервалів. Звичайний `time.time()` для цього не годиться.

Крок 2. Запустити: `python benchmark.py`. Результати збережуться у `results.csv`

Крок 3. Окремо виміряти холодний і теплий старт локальної моделі:

```
ollama stop llama3.2:3b
```

Одразу запустити `benchmark.py` і записати TTFT короткого промпту — це **холодний старт**. Запустити ще раз без зупинки моделі — це **теплий**.

Крок 4. Заповнити таблиці:

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна		короткий			
локальна		середній			
локальна		довгий			
хмарна		короткий			
хмарна		середній			
хмарна		довгий			

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с		
Всього, с		

Крок 5. Побудувати стовпчикову діаграму (matplotlib): швидкість у символах за секунду для всіх шести комбінацій «модель × промпт». Можете запитати у LLM - як написати код для генерації стовпчикової діаграми.

Крок 6. Відповісти на питання (по 1–2 речення):

1. У якій моделі кращий TTFT і чому саме в неї?
2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.
3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?
4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?
5. Чому холодний старт повільніший за теплий?

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?
2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?
3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?
4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?
5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

У звіт додати дані таблиць, діаграму та стислі відповіді на питання (2–3 речення на кожну). Стисло напишіть свої думки: який із вимірних показників важливіший

для інтерактивного чат-бота, а який — для фонові обробки великих обсягів тексту, і чому.

Крок 6. Оцінити якість роботи моделей

У попередніх завданнях ми дізналися, яка модель швидша. Але швидкість сама по собі нічого не варта: модель, яка миттєво видає дурню, гірша за ту, що думає п'ять секунд і відповідає правильно.

Проблема в тому, що «якість» — поняття розпливчате. Запис у звіті «модель А здалася розумнішою» — це не результат, а враження: його неможливо перевірити й неможливо відтворити.

Інженерне рішення — оцінювання за рубрикою: фіксований набір завдань, однаковий прогін через усі моделі, бали за заздалегідь визначеними правилами. Наприкінці курсу ви робитимете те саме для агентів.

Правило, яке не можна порушувати: **набір завдань складається до того, як ви побачили відповіді**. Підганяти завдання під результат — головний ворог чесного вимірювання.

Ще один нюанс: **модель недетермінована, на повторний запуск того самого завдання вона може відповісти інакше**. Тому кожне завдання прогоняється двічі. Розбіжність між прогонами — це не збій, а характеристика моделі, така сама важлива, як швидкість.

Використанні завдання

Кожне завдання ловить окремий тип поведінки моделі, і разом вони дають профіль її сильних і слабких місць. Нижче — що саме перевіряє кожне і на що дивитися при оцінюванні.

Завдання 1. Фактичні знання.

Що перевіряємо: чи знає модель факт і, головне, чи зізнається вона, що не знає.

Мовна модель не має бази даних, у якій можна подивитися відповідь. Вона генерує текст, який виглядає правдоподібно, спираючись на статистику мільярдів прочитаних текстів. Якщо потрібного факту в цій статистиці не було або він був представлений слабо, модель не мовчить — вона видає найправдоподібніший варіант. Це і називається галюцинацією: відповідь звучить упевнено й правильно оформлена, але вигадана.

Саме тому вам обов'язково треба знайти правильну відповідь в авторитетному джерелі, без цього ви не зможете відрізнити знання від впевненої вигадки, а сама модель вам не підкаже — вона однаково впевнена в обох випадках.

На що дивитися? Найпоказовіше — прогнати завдання двічі. Якщо модель обидва рази дала той самий рік, вона, найімовірніше, справді його знає. Якщо роки різні — вона їх вигадує, і навіть випадковий збіг із правильною відповіддю не рахується за знання.

Локальні моделі на 3–4 млрд параметрів особливо часто плавають на регіональних фактах: даних про українські установи в їхньому навчанні було мало.

Практичний висновок для курсу: саме через галюцинації існує RAG – підхід, коли моделі підсовують потрібні документи в контекст замість того, щоб покладатися на її пам'ять.

Завдання 2. Дотримання формату.

Що перевіряємо: чи можна використати модель як компонент програми, а не як співрозмовника?

Це найважливіше завдання набору з практичного погляду. Коли модель відповідає людині, невеликий безлад у форматі не заважає — людина зрозуміє. Але коли відповідь моделі йде далі в код, вона має бути машинозчитуваною. Один зайвий символ — і `json.loads()` падає з помилкою, а вся програма зупиняється.

Проблема в тому, що модель навчена на текстах, де код і JSON прийнято обгортати в markdown-блоки `json`. Ця звичка настільки міцна, що часто перемагає вашу пряму інструкцію «без markdown». Модель ніби «знає краще», як має виглядати правильно оформлена відповідь.

На що дивитися? Тут оцінка об'єктивна, суб'єктивності немає взагалі: пропустить відповідь через `json.loads()`. Парситься одразу — 2 бали, треба зрізати обгортку — 1, не парситься — 0. Типові дефекти: markdown-обгортка, вступна фраза «Ось JSON:», кома після останнього поля, одинарні лапки замість подвійних.

Практичний висновок: саме тому в лабораторній 2 ви будете працювати не з «проханням дати JSON», а зі схемами Pydantic і повторними запитами при невдачі. Промпт — ненадійний спосіб керувати форматом, і ви маєте побачити це на власних числах, перш ніж вивчати надійні.

Завдання 3. Робота з наданим текстом.

Що перевіряємо: чи вміє модель працювати з інформацією, яку ви їй дали, не підмішуючи те, що вона «пам'ятає».

Це принципово інша задача, ніж перші дві. Там модель діставала відповідь із себе, тут — з тексту в запиті. Друге виходить у моделей значно надійніше, і на цьому побудована більшість реальних застосувань: підсумовування документів, витяг даних, відповіді на питання по базі знань.

Але й тут є вразливість — модель може непомітно спотворити дані. Замінити «25 тисяч» на «близько 20 тисяч», округлити відсоток, переплутати два роки місцями. У переказі це виглядає природно, і саме тому непомітно.

Навіщо потрібні числа в тексті? Це пастка на таке спотворення. Число або збігається з оригіналом, або ні — перевірка займає секунди й не потребує суб'єктивної оцінки. Без чисел довелося б на око вирішувати, «чи добре стиснуто», а це знову враження замість вимірювання.

Який текст шукати? Абзац на 5–6 речень за темою вашого варіанта, у якому є щонайменше три конкретні числа: роки, кількості, відсотки, суми. Джерело — Вікіпедія, офіційний сайт установи, новинна стаття. Для варіанта 1 (освіта) підійде абзац про університет із роком заснування, кількістю факультетів і кількістю студентів.

На що дивитися? 2 бали — два речення, усі числа на місці. 1 бал — числа правильні, але речень більше або додано те, чого в тексті не було. 0 балів — хоча б одне число спотворене чи втрачене.

Завдання 4. Генерація коду.

Що перевіряємо: чи придатний вивід моделі до безпосереднього запуску.

Код зручний для оцінювання тим, що він або працює, або ні — суб'єктивності мінімум. Але тут перевіряється не тільки правильність алгоритму, а й дві додаткові речі.

Перша — чи виконано умову повністю. У завданні є конкретна вимога: ігнорувати пробіли й регістр, відкинути найнижчу оцінку, порахувати з категорією. Модель може написати робочу функцію, яка цю вимогу не реалізує. Це типовий і небезпечний випадок: код запускається, тестів немає, помилку помічають пізно.

Друга — знову формат. Ви написали «лише код», а модель, найімовірніше, додасть пояснення до й після. Тут це не критично, але показує ту саму закономірність, що й у завданні 2: інструкції щодо формату виконуються ненадійно.

На що дивитися? Скопіюйте функцію, запустіть на 2–3 прикладах. 2 бали — працює й виконує всю умову. 1 бал — працює, але умову виконано частково, або навколо коду багато зайвого тексту. 0 балів — не запускається чи дає неправильний результат.

Завдання 5. Міркування в кілька кроків.

Що перевіряємо: чи здатна модель вивести відповідь, якої немає в готовому вигляді.

Це найважче завдання набору, і причина в самій механіці генерації. Модель видає текст по одному токеноу за раз. Коли ви просите одразу назвати число, усі проміжні обчислення мають вміститися в один крок — а це фізично обмежує складність задачі, яку модель витягне.

Задачі у варіантах спеціально влаштовані так, що перша інтуїтивна відповідь неправильна. У задачі про 25 студентів напрошується $25 - 12 - 9 = 4$, і модель часто саме це й пише — забувши, що четверо порохувалися двічі.

Ось чому в умові стоїть «покажи хід розв’язання». Коли модель розписує кроки, кожен проміжний результат стає текстом у контексті, на який спираються наступні кроки. Вона фактично використовує власний вивід як робочу пам’ять — і точність помітно зростає.

На що дивитися? Тут можливі два різні провали, і їх треба розрізняти. Перший: правильна відповідь без показаного ходу — інструкцію не виконано, 1 бал. Другий: розписаний хід, який виглядає переконливо, але містить арифметичну помилку — 0 балів. Другий випадок цінніший для звіту, бо демонструє, що правдоподібність міркування і його правильність — різні речі. Якщо все вірно - ставимо 2 бали.

Саме на цьому завданні найяскравіше видно різницю між міркувальною та інструктивною моделлю, яку ви вимірювали раніше.

Завдання 6. Якість української мови.

Що перевіряємо: те, що частіше за все ігнорують і потім наступають на це в реальному проєкті.

Мовні моделі вчилися переважно на англomовних текстах. Українських даних у навчанні на порядки менше, а російських — помітно більше, ніж українських. Наслідок передбачуваний: модель формально відповідає українською, але текст рясніє русизмами й кальками, іноді зривається на іншу мову посеред абзацу, а термінологію вживає неточно.

Це не косметична проблема. Якщо ви будете систему для українських користувачів, якість мови — така сама вимога, як швидкість чи точність. І що менша модель, то гірше вона з цим справляється.

Зверніть увагу на цікавий ефект, який ви, найімовірніше, побачите: модель може думати однією мовою, а відповідати іншою. Ви це вже спостерігали, коли міркувальна модель розписувала хід розв’язання російською чи англійською, а фінальну відповідь давала українською.

На що дивитися. 2 бали — природна українська, коректна термінологія, обидва абзаци по суті. 1 бал — суть правильна, але є русизми, кальки, кострубаті конструкції. 0 балів — відповідь не українською або перехід на іншу мову посеред тексту.

Як ці шість завдань працюють разом

Кожне окремо не дає повної картини — разом вони показують профіль моделі. Типовий результат, який ви побачите:

- мала локальна модель непогано справляється із завданнями 3 і 4 (робота з наданим текстом і простий код), провалює 1 і 5 (факти й міркування), плаває на 2 і 6 (формат і мова);
- хмарна велика модель упевнено бере 1, 3, 4, 6, але може так само проігнорувати вимогу «без markdown» у завданні 2 — і це важливе спостереження: розмір моделі не розв’язує проблему надійності формату.

Саме тому висновок «модель А краща за модель Б» майже завжди беззмістовний. Правильне формулювання — під яку задачу яка модель. І в наступних лабораторних, обираючи модель для агента, ви дивитиметесь передусім на завдання 2: агент викликає інструменти через структуровані дані, і модель, яка не тримає формат, зламає всю систему.

Завдання 5. Оцінювання якості моделей за рубрикою

Шкала оцінювання:

Таблиця 9: Загальна шкала оцінювання

Бал	Критерій
0	Не впорався: неправильна відповідь, зламаний формат, не та мова, відмова відповісти
1	Частково: суть правильна, але є дефект — зайвий текст навколо JSON, помилка в дрібниці, кострубата мова, відсутній хід розв’язання там, де його просили
2	Виконано повністю

Таблиця 10: Критерії оцінювання за завданнями

№	2 бали	1 бал	0 балів
1	Відповідь збігається з авторитетним джерелом, обидва прогони однакові	Правильна відповідь, але прогони розійшлися	Відповідь не збігається з джерелом
2	<code>json.loads()</code> парсить без правок	Парситься після видалення markdown-обгортки або вступної фрази	Не парситься взагалі
3	Два речення, усі числа збережені без спотворень	Числа правильні, але речень більше двох або додано інформацію, якої не було в тексті	Хоча б одне число спотворене або втрачене
4	Код запускається і виконує всю умову завдання	Код працює, але умову виконано частково, або навколо коду багато зайвого тексту	Не запускається або дає неправильний результат
5	Правильна відповідь і показаний хід без помилок	Правильна відповідь без ходу, або вірний підхід із дрібною помилкою в обчисленнях	Неправильна кінцева відповідь
6	Природна українська, коректна термінологія, обидва абзаци по суті	Суть правильна, але є русизми, кальки, кострубаті конструкції	Відповідь не українською або перехід на іншу мову посеред тексту

Максимум для однієї моделі — 12 балів. Для завдання 2 діє об'єктивний тест: `json.loads()` на відповіді моделі. Парситься без правок — 2 бали. Треба вирізати обгортку `json` — 1 бал. Не парситься — 0.

Варіанти (номер у журналі):

Таблиця 11: Варіанти: завдання 1 і 2

№	Тема	Завдання 1 (факт)	Завдання 2 (формат)
1	Освіта	Рік заснування Одеського національного університету імені І. І. Мечникова	JSON: name, year, city для цього університету
2	Транспорт	Рік відкриття Київського метрополітену	JSON: city, year, lines_count для Київського метро
3	Медицина	Рік відкриття пеніциліну Флемінгом	JSON: discovery, year, scientist
4	Спорт	Рік перших сучасних Олімпійських ігор	JSON: event, year, city
5	Фінанси	Рік введення гривні як національної валюти	JSON: currency, code, year
6	Екологія	Рік підписання Паризької кліматичної угоди	JSON: agreement, year, city
7	Кіно	Рік першого публічного кінопоказу братів Люм'єр	JSON: event, year, city
8	Туризм	Рік відкриття нинішньої будівлі Одеського оперного театру	JSON: landmark, city, year
9	Астрономія	Рік першого польоту людини в космос	JSON: event, year, person
10	Техніка	Рік випуску першого iPhone	JSON: product, year, company

Таблиця 12: Варіанти: завдання 4, 5 і 6

№	Завдання 4 (код)	Завдання 5 (логіка)	Завдання 6 (українська)
1	<code>is_palindrome(s)</code> , ігнорує пробіли й регістр	25 студентів, 12 знають Python, 9 Java, 4 обидві. Скільки не знають жодної?	Чим стек відрізняється від черги
2	<code>total_weight(items)</code> — сума wag зі списку словників	120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів?	Чим контейнерне перевезення відрізняється від навалочного
3	<code>bmi(weight_kg, height_m)</code> з категорією	40 пацієнтів, 18 з температурою, 15 з кашлем, 7 з обома. У скількох немає симптомів?	Чим вірус відрізняється від бактерії
4	<code>win_rate(results)</code> зі списку "W"/"L"/"D"	8 команд, кожна з кожною по разу. Скільки матчів? При 4 матчах на день — скільки днів?	Чим спринт відрізняється від стаєрського бігу
5	<code>compound_interest(principal, rate, years)</code>	Депозит 20000 грн під 12% річних, податок 19,5% з відсотків. Скільки на руках через рік?	Чим акція відрізняється від облігації
6	<code>sort_waste(items)</code> за словником категорій	500 т сміття: 40% органіка, 25% пластик. Переробляють 60% пластику. Скільки тонн пластику на полігон?	Чим відновлювані джерела енергії відрізняються від невідновлюваних
7	<code>top_rated(films, n)</code> — n найкращих за рейтингом	Зал 180 місць, сеанс 1 год 45 хв + 15 хв перерва, робота з 10:00 до 23:00. Скільки сеансів і глядачів?	Чим документальне кіно відрізняється від художнього
8	<code>trip_cost(days, daily_cost, transport)</code>	Поїздка 6 днів: житло 900/добу, їжа 500/день, дорога 2400. Троє, житло й дорога порівну. Скільки заплатить кожен?	Чим хостел відрізняється від готелю
9	<code>light_travel_time(distance_km)</code>	Зонд 60000 км/год, відстань 480 млн км. Скільки повних діб?	Чим зоря відрізняється від планети
10	<code>average_grade(grades)</code> без найнижчої оцінки	Сервер обробляє 1200 запитів/хв, 15% дають помилку, з них 40% повторюються успішно. Скільки запитів втрачено за годину?	Чим оперативна пам'ять відрізняється від жорсткого диска

Текст для завдання 3: студент знаходить сам, 5–6 речень за темою свого варіанта, обов'язково з числами. Формулювання: «Стисни до двох речень, збережи всі числа.»

Крок 1. Створити файл `quality.py`

```
from providers import make_client
import json
```

```

TEXT = "<текст 5-6 речень за темою вашого варіанта>"

TASKS = {
    "1_fact": "<завдання 1 вашого варіанта>. Відповідай лише роком.",
    "2_format": "<завдання 2 вашого варіанта>. Без пояснень, без markdown, лише JSON.",
    "3_summary": "Стисни наступний текст до двох речень, збережи всі числа: \n\n" + TEXT,
    "4_code": "<завдання 4 вашого варіанта>. Лише код.",
    "5_logic": "<завдання 5 вашого варіанта>. Покажи хід розв'язання.",
    "6_language": "<завдання 6 вашого варіанта>. Поясни українською, два абзаци.",
}

results = {}

for provider in ["local", "cloud"]:
    client, model = make_client(provider)
    results[model] = {}
    for task_id, prompt in TASKS.items():
        answers = []
        for attempt in range(2): # два прогони на завдання
            r = client.chat.completions.create(
                model=model,
                messages=[{"role": "user", "content": prompt}],
                temperature=0.7,
            )
            answers.append(r.choices[0].message.content)
        results[model][task_id] = answers
        print(f"{model} / {task_id} - готово")

with open("quality.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)

```

Крок 2. Запустити для всіх моделей: `llama3.2:3b`, `qwen3:4b`, хмарна. Відповіді збережуться у `quality.json`.

Крок 3. Знайти правильну відповідь на завдання 1 в авторитетному джерелі. Без цього неможливо визначити, чи галюцинує модель.

Крок 4. Перевірити завдання 2 через `json.loads()` для кожної моделі.

Крок 5. Відкрити `quality.json`, прочитати відповіді, виставити бали. Заповнити таблиці.

Таблиця 13: Оцінка якості за рубрикою

Завдання	Лlama3.2:3b	qwen3:4b	Хмарна модель
1. Факт			
2. Формат			
3. Стиснення			
4. Код			
5. Логіка			
6. Українська			
Разом /12			

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	Лlama3.2:3b	qwen3:4b	Хмарна модель
1. Факт			
2. Формат			
3. Стиснення			
4. Код			
5. Логіка			
6. Українська			

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?
2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?
3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?
4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Крок 7. Обов'язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

- повний текст запиту;
- повний текст відповіді, без скорочень;
- яка правильна відповідь мала бути;
- пояснення, чому так сталося.

Пояснення не повинно бути «модель тупа». Вигадала факт, якого не знала? Проігнорувала інструкцію про формат, бо звикла обгортати код у markdown? Загубилась у багатокроковій арифметиці? Перейшла на англійську, бо українських текстів у навчанні було мало?

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п'ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?
3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?
4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?
5. Що таке галюцинація мовної моделі і чому модель радше вигидає факт, ніж визнає незнання?

У звіт додати дані таблиць, дані Кроку 7, стислі відповіді на питання (2–3 речення на кожну). Стисло напишіть свої думки: наскільки надійною виявилася оцінка за рубрикою, які завдання виявились найпоказовішими для порівняння моделей, і яку модель ви обрали б для роботи з українськомовним текстом.

Крок 7. Дослідити вплив `temperature` і `top_p`

Модель не «пише текст». На кожному кроці вона рахує ймовірність для кожного можливого наступного токена. Наприклад, після фрази «Столиця України – » розподіл виглядає приблизно так:

- «Київ» → 0.94
- «місто» → 0.03
- «Одеса» → 0.01
- ...ще тисячі варіантів з мізерними ймовірностями

Далі треба обрати один. За цей вибір і відповідають два параметри.

Temperature змінює контрастність розподілу перед вибором. При **temperature** = 0 завжди береться найімовірніший варіант — «Київ», щоразу. При підвищенні температури розподіл «сплющується»: різниця між імовірним і малоімовірним зменшується, і з'являється шанс обрати щось несподіване. При **temperature** = 1.5 модель цілком може написати «Одеса».

Top_p діє інакше — він обмежує коло кандидатів. При **top_p** = 0.1 модель бере лише найімовірніші варіанти, які разом набирають 10% усієї ймовірності, а решту викидає з розгляду. При **top_p** = 1.0 розглядаються всі. Цей параметр не змінює співвідношення ймовірностей, він відсікає «хвіст».

Головне, що треба перевірити на досвіді: універсально правильного значення не існує. Для фактичних питань і видачі JSON потрібна низька температура — вам потрібна передбачуваність. Для генерації ідей і назв потрібна висока — інакше модель щоразу видає ту саму нудну відповідь.

Завдання 6. Керування випадковістю: `temperature` і `top_p`

Варіанти (номер у журналі):

Таблиця 15: Варіанти завдань

№	Творче завдання	Фактичне питання
1	Назва стартапу в галузі логістики	Рік заснування Національного університету «Одеська Політехніка»
2	Назва застосунку нагадувань про ліки	Рік відкриття Київського метрополітену
3	Назва платформи взаємного навчання студентів	Рік відкриття пеніциліну Флемінгом
4	Назва сервісу доставки домашньої їжі	Рік перших сучасних Олімпійських ігор
5	Назва застосунку пошуку партнерів для тренувань	Рік введення гривні
6	Назва освітнього проєкту з астрономії	Рік підписання Паризької кліматичної угоди
7	Назва застосунку обліку особистих витрат	Рік першого кінопоказу братів Люм'єр
8	Назва сервісу обміну речами	Рік першого польоту людини в космос
9	Назва рекомендаційного сервісу фільмів	Рік випуску першого iPhone
10	Назва застосунку пішохідних екскурсій	Рік відкриття будівлі Одеського оперного театру

Крок 1. Створити файл `sampling.py` :

```

from providers import make_client

client, model = make_client("local")

CREATIVE = "<творче завдання вашого варіанта>"
FACTUAL = "<фактичне питання вашого варіанта>"

def run(prompt, n=5, **params):
    """Запускає промпт n разів і повертає список відповідей."""
    answers = []
    for _ in range(n):
        r = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            **params,
        )
        answers.append(r.choices[0].message.content.strip())
    return answers

```

Крок 2. Творче завдання, вплив `temperature`:

```
for t in [0.0, 0.7, 1.5]:
    answers = run(CREATIVE, temperature=t)
    print(f"temperature={t}: унікальних {len(set(answers))} з 5")
    for a in answers:
        print("    ", a)
    print()
```

Ключова метрика — кількість унікальних відповідей із п'яти. Це число йде в таблицю. Очікуваний результат: при 0.0 одна й та сама назва п'ять разів, при 1.5 — п'ять різних, але серед них можуть трапитись безглузді.

Крок 3. Творче завдання, вплив top_p .

Той самий цикл, але температура фіксована, змінюється top_p :

```
for p in [0.1, 0.5, 1.0]:
    answers = run(CREATIVE, temperature=0.7, top_p=p)
    print(f"top_p={p}: унікальних {len(set(answers))} з 5")
```

Порівняйте з попереднім кроком: обидва параметри впливають на різноманітність, але чи однаково? Дивіться на характер відповідей, не лише на їх кількість.

Крок 4. Фактичне питання.

Найважливіший експеримент. Прогоніть фактичне питання при temperature= 0.0 і temperature = 1.5 , по 5 разів:

```
for t in [0.0, 1.5]:
    answers = run(FACTUAL, temperature=t)
    print(f"temperature={t}:", answers)
```

При високій температурі ви з великою ймовірністю побачите різні роки. Модель вигадуватиме. Це і є демонстрація того, що температура — не «регулятор розумності», а регулятор випадковості, і на фактичних задачах вона працює проти вас.

Правильну відповідь знайдіть в авторитетному джерелі, посилання вкажіть у звіті.

Крок 5. Перевірка відтворюваності.

```
answers = run(FACTUAL, n=10, temperature=0.0)
print("Усі відповіді однакові:", len(set(answers)) == 1)
```

Швидше за все збіжаться. Але гарантії немає – на це впливають особливості обчислень з плаваючою комою й паралелізація. Тримайте це в голові: у наступних лабораторних, порівнюючи дві версії агента, частина різниці може бути просто шумом, а не наслідком ваших покращень.

Крок 6. Заповнити таблиці:

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0		
temperature	0.7		—
temperature	1.5		
top_p	0.1		—
top_p	0.5		—
top_p	1.0		—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	
Усі відповіді однакові (так / ні)	

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?
2. Яке — генератору варіантів заголовків?
3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?
4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?
5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?
2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?
3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?
4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?
5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

У звіт додати дані таблиць, стислі відповіді на питання (2–3 речення на кожну). Стисло напишіть свої думки: які значення параметрів ви оберете для наступних лабораторних, де модель має повертати структуровані дані, і чому налаштування «за замовчуванням» може виявитись непридатним.

Крок 8. Звести все разом і зробити висновок

Заповніть підсумкову таблицю:

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В/8В)	Хмарна
Швидкість генерації			
TTFT			
Якість, балів /12			
Стабільність формату (завдання 2)			
Потрібні ресурси			
Вартість			
Куди йдуть ваші дані			
Працює без інтернету			
Коли обирати саме її			

Останній рядок – головний, і його не можна заповнити одним словом. Від вас потрібне правило вибору: за яких умов ви візьмете локальну модель, попри її слабкість, а за яких — хмарну, попри залежність від мережі й передачу даних назовні.

Що здаємо

Репозиторій на GitHub:

```
lab1/
├─ report.pdf          звіт: конфігурація, усі таблиці, графік, висновки
├─ providers.py        налаштування провайдерів
├─ benchmark.py        заміри швидкості
├─ quality.py          прогін завдань на якість
├─ sampling.py         експерименти з temperature / top_p
├─ results.csv         сирі дані замірів
├─ quality.json        збережені відповіді моделей
├─ .gitignore          з рядком .env
└─ (файл .env у репозиторій НЕ потрапляє)
```